

Dossier de présentation technique approfondi du projet Perseo

Executive summary

Perseo, aussi désigné dans vos échanges comme **You Money Flow**, doit être traité techniquement comme une plateforme de **gestion financière personnelle à noyau comptable** : collecte de données bancaires, normalisation, classification, génération d'écritures équilibrées, projection de trésorerie, suivi patrimonial, gestion d'abonnements et pilotage analytique. Dans cette logique, le cœur du système ne doit pas être conçu comme une simple application de "budgeting", mais comme un **système de vérité financière** fondé sur des règles d'intégrité, de traçabilité et de recalcul déterministe. Cette orientation est cohérente avec les capacités attendues du projet et avec les caractéristiques des briques techniques visées : Supabase apporte un socle PostgreSQL, des fonctions base de données, du RLS et des Edge Functions ; Firebase apporte Auth, Firestore, Storage, Functions et Hosting ; Bridge fournit l'agrégation bancaire via API REST/OAuth et Connect ; Stripe couvre la facturation SaaS ; OpenAI doit rester cantonné à des fonctions assistées, non décisionnelles ni "source of truth". ¹

Le point structurant de l'architecture est le suivant : **les données financières critiques doivent résider dans un modèle relationnel et transactionnel**, ce qui conduit, par inférence d'architecture, à recommander **Supabase/PostgreSQL comme source de vérité primaire** pour le Ledger V2, les règles de calcul, les contrôles d'intégrité, les rapprochements et les prévisions. Firebase peut rester présent comme provider alternatif ou secondaire pour certains usages déjà investis — authentification, hébergement, stockage documentaire, prototypes ou certains services événementiels — mais le noyau financier, lui, est plus naturellement porté par PostgreSQL et ses fonctions côté base. Cette recommandation découle des propriétés officielles des deux plateformes : Postgres/RLS/fonctions côté base d'un côté, Firestore/règles de sécurité/document store de l'autre. ²

L'état d'accès aux sources GitHub impose toutefois une réserve méthodologique importante : le dépôt demandé, `Ulysse03/your-money-flow`, **n'est pas accessible publiquement depuis le web au moment de la recherche** et renvoie une erreur HTTP 404 ; en conséquence, **aucun commit SHA, chemin de fichier ou issue vérifiable publiquement n'a pu être cité dans ce dossier**. Le présent document est donc rédigé comme un **cahier des charges technique cible**, fondé sur votre brief fonctionnel et sur les documentations officielles des fournisseurs cités, prêt pour une revue d'architecture et une implémentation, mais non pour un audit "as-built" du code existant. ³

La décision d'architecture prioritaire est donc claire : **front web modulaire, BackendManager** abstrait supportant **Supabase primary / Firebase secondary ou fallback, Edge Functions/Cloud Functions** pour tous les flux secrets et webhooks, **Bridge** pour la collecte bancaire, **Ledger V2** comme noyau comptable, **prévisionnel dérivé du ledger et des engagements, CI/CD GitHub Actions + hébergement Vercel ou Firebase Hosting, sauvegardes, chiffrement, consentement et audit** de bout en bout. Supabase documente les migrations, les sauvegardes et le PITR ; Vercel documente le déploiement continu GitHub et les environnements preview ; Firebase documente l'émulation locale, l'export/import Firestore et le hosting ; Stripe et Bridge documentent respectivement les webhooks, la sécurité et les flux d'intégration. ⁴

Hypothèses, objectifs techniques et cas d'usage

Le dossier se limite strictement aux **points techniques** demandés : objectifs, besoins, cas d'usage, architecture du site, flux, sources, modèles de données, règles de calcul, API, intégrations, sécurité, conformité, gouvernance, migration et tests. Les paramètres non explicités dans votre brief sont maintenus comme **non spécifié** : budget, SLA business contractuel, volumétrie cible d'utilisateurs finaux, choix backend définitivement arbitré, chiffage de run mensuel, multi-région, PRA contractuel. Cette distinction est nécessaire pour éviter de transformer des hypothèses d'architecture en exigences arrêtées.

Le projet doit viser les objectifs techniques suivants : centraliser les flux bancaires et manuels, produire un registre financier cohérent, permettre le recalcul complet des agrégats à partir des sources, séparer strictement **données sources**, **données dérivées** et **vues analytiques**, et limiter l'IA à des propositions annotées avec score de confiance et validation humaine. Cette orientation est directement conforme aux principes de sécurité, de minimisation et de protection "by design" exigés par le RGPD, ainsi qu'au besoin d'authentification forte et de gestion de consentement du périmètre Open Banking/PSD2 exploité via Bridge. ⁵

Les cas d'usage techniques structurants sont les suivants : import bancaire initial et incrémental, import CSV/manuel, classification assistée, détection de doubles et de transferts internes, génération d'écritures ledger, rapprochement d'abonnements, calcul de résultat/trésorerie/patrimoine, projection future, exports réglementaires ou utilisateurs, audit des traitements, réinitialisation/rejeu d'un périmètre et restauration après incident. Bridge documente précisément les notions d'items, comptes, transactions, synchronisation incrémentale via `since`, statuts d'items et expiration de l'authentification forte, ce qui alimente directement ces cas d'usage d'ingestion. ⁶

L'architecture d'information interne du site doit être alignée sur les grands domaines suivants, qui structurent aussi les routes, les composants et les permissions : **Pilotage**, **Flux**, **Ledger**, **Prévisionnel**, **Patrimoine**, **Engagements**, **Fiscalité**, **Administration**. Une arborescence web cohérente est recommandée ci-dessous.

Domaine	Routes recommandées	Source de vérité	Données dérivées principales
Pilotage	<code>/dashboard</code> , <code>/analytics</code> , <code>/alerts</code>	Vues matérialisées / vues SQL	KPI, scorecards, anomalies
Flux	<code>/transactions</code> , <code>/accounts</code> , <code>/imports</code> , <code>/merchants</code>	transactions, accounts, imports	catégories, suggestions IA
Ledger	<code>/ledger/journal</code> , <code>/ledger/entries</code> , <code>/ledger/rebuild</code>	ledger_entries, ledger_lines	balance, contrôle d'intégrité
Prévisionnel	<code>/forecast</code> , <code>/scenarios</code> , <code>/calendar</code> , <code>/future-journal</code>	forecast_scenarios, forecast_entries	solde projeté, stress tests

Domaine	Routes recommandées	Source de vérité	Données dérivées principales
Patrimoine	<code>/wealth/assets</code> , <code>/wealth/loans</code> , <code>/wealth/net-worth</code>	assets, loans, patrimoine_entries	valeur nette, amortissements
Engagements	<code>/contracts</code> , <code>/subscriptions</code> , <code>/obligations</code>	contracts, subscriptions, occurrences	échéanciers, rapprochements
Fiscalité	<code>/tax</code> , <code>/tax/candidates</code> , <code>/tax/export</code>	tax_items, tax_candidates	dossier fiscal, contrôles
Admin	<code>/admin/modules</code> , <code>/admin/providers</code> , <code>/admin/security</code>	module_registry, asset_packs, audit_logs	états, versioning, conformité

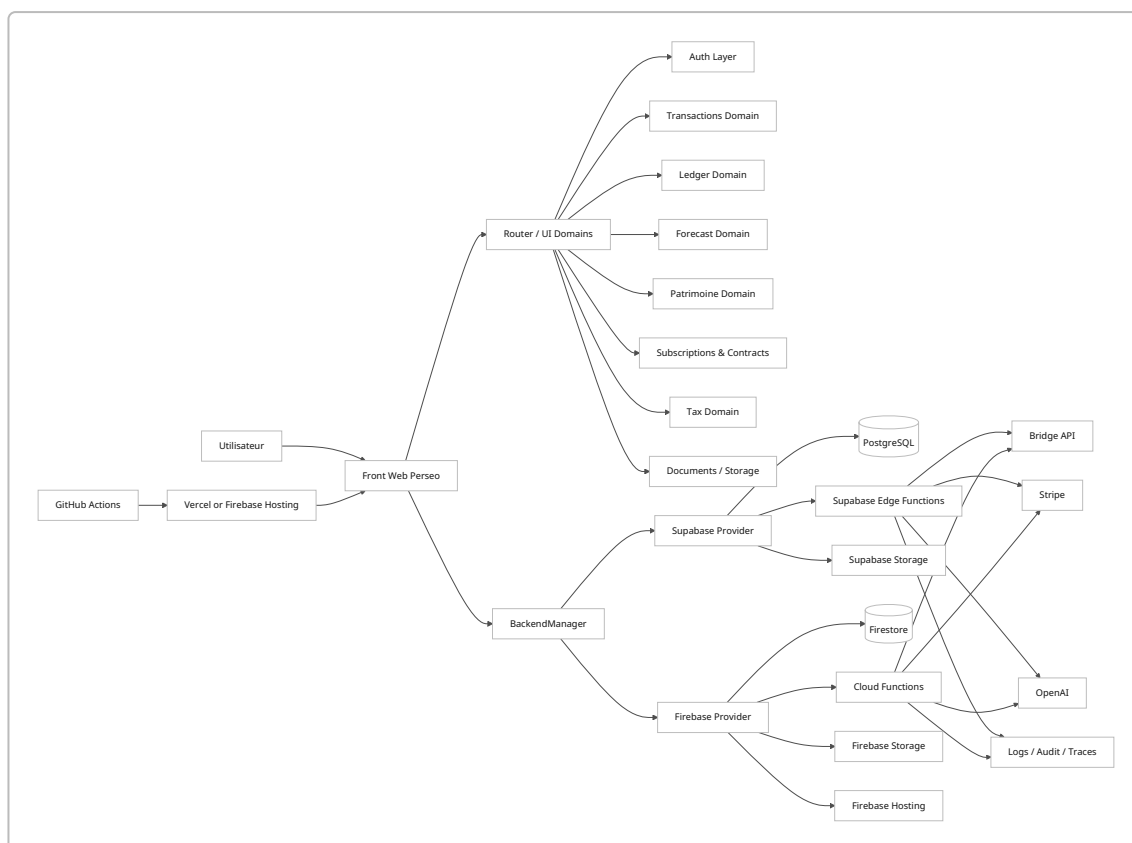
Les exigences non fonctionnelles cibles doivent être explicites dès le cahier des charges : disponibilité applicative cible, latence, RTO, RPO, sauvegarde, chiffrement, traçabilité, qualité de service des intégrations externes, observabilité et réversibilité. Les fournisseurs documentent des mécanismes mobilisables — Supabase daily backups et PITR, Firestore export/import planifiable, Vercel preview/production, HTTPS/TLS par défaut, webhooks Stripe et Bridge, limites et bonnes pratiques OpenAI — mais les objectifs chiffrés métier doivent être définis côté projet car ils ne sont pas spécifiés à ce stade.

7

Architecture cible du système

L'architecture recommandée est une architecture en couches, avec **front web**, **backend abstraction layer**, **provider data layer**, **compute serverless**, **integrations externes** et **observabilité/CI-CD**. Le front peut être servi en SPA ou en framework hybride moderne ; le point central n'est pas le framework, mais le fait que toutes les opérations sensibles soient **déportées côté serveur** et qu'aucun secret Bridge, Stripe ou OpenAI ne soit exposé au navigateur. Stripe recommande une architecture sécurisée et rappelle que la conformité PCI est une responsabilité partagée ; OpenAI recommande de conserver les clés en environnement sûr ; Bridge recommande la sécurisation des webhooks et l'autorisation IP côté serveur.

8



L'arbitrage entre Supabase et Firebase peut être formalisé comme suit. Il ne s'agit pas d'un choix purement "cloud provider", mais d'un choix de **modèle transactionnel**, de **style de requête** et de **contrôle d'intégrité**. Pour un produit financier personnel avec ledger, règles d'équilibrage et recalculs analytiques SQL, la recommandation reste d'utiliser Supabase/Postgres comme primaire. ⁹

Critère	Supabase / PostgreSQL	Firebase / Firestore	Recommandation Perseo
Modèle	Relationnel SQL + fonctions DB + RLS ¹⁰	Document store + Security Rules ¹¹	Supabase pour le cœur financier
Intégrité comptable	Forte, naturelle via contraintes, transactions SQL, triggers, vues	Possible, mais plus applicative et moins naturelle pour le double-entry	Supabase primaire
Calculs analytiques	Très adaptés aux agrégations et vues	Faisables mais moins ergonomiques pour reporting financier relationnel	Supabase primaire
Fonctions serveur	Edge Functions Deno, globales ¹²	Cloud Functions HTTP/événements ¹³	Deux options possibles
Stockage fichiers	Supabase Storage avec contrôle fin ¹⁴	Firebase Storage avec Auth/Rules ¹⁵	Selon existant

Critère	Supabase / PostgreSQL	Firebase / Firestore	Recommandation Perseo
Sauvegarde / reprise	Backups quotidiens + PITR ; Storage hors backup DB ¹⁶	Export/import Firestore, exports planifiables ¹⁷	Supabase plus simple pour DB primaire
Hébergement	Front via Vercel / autre ; Supabase n'est pas un front host principal	Firebase Hosting / App Hosting ¹⁸	Vercel ou Firebase Hosting
Local dev / tests	CLI, migrations, debug Edge Functions ¹⁹	Local Emulator Suite ²⁰	Dual-tooling possible
Choix final	Non spécifié	Non spécifié	Décision recommandée : Supabase primaire

L'architecture de déploiement cible doit rester simple et industrialisable : **GitHub** comme source de code, **GitHub Actions** pour CI/CD, **Vercel** pour le front web si l'objectif est la rapidité des previews et la gestion simple des branches, ou **Firebase Hosting/App Hosting** si l'écosystème Firebase reste dominant. GitHub Actions documente le CI/CD et les matrices d'exécution ; Vercel documente les preview deployments automatiques, les promotions vers la production et le HTTPS/TLS ; Firebase Hosting documente l'hébergement web sécurisé sur CDN global. ²¹

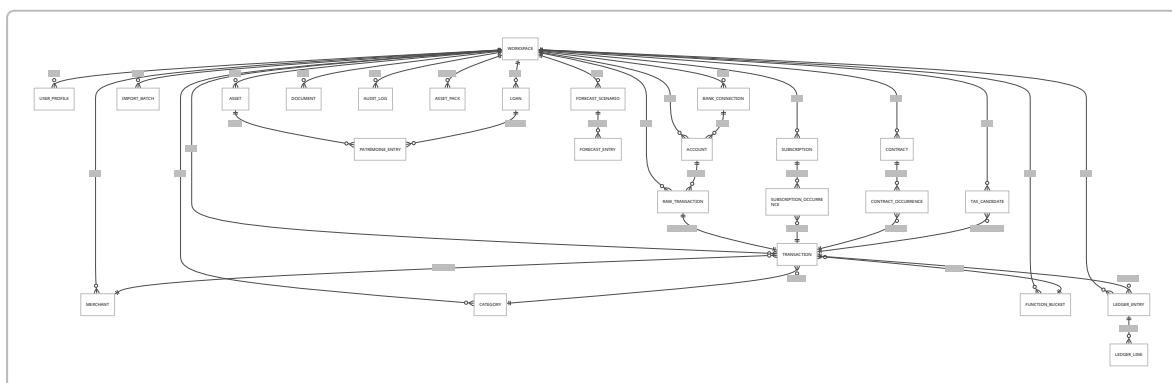
Une chaîne CI/CD technique recommandée est la suivante :

lint → tests unitaires → tests de schéma/migrations → tests Edge Functions / Cloud Functions → tests E2E → build front → déploiement preview → smoke tests → promotion prod. Supabase documente la gestion des migrations via CLI ; Firebase l'émulation locale ; Stripe les webhooks de test et test clocks ; GitHub Actions la stratégie de matrices. ²²

Modèle de données, gestion des données et règles de calcul

Le principe cardinal de modélisation est de distinguer **données sources**, **données normalisées**, **ledger**, **données métiers dérivées**, **prévisionnel**, **référentiels** et **audit**. L'import bancaire Bridge fournit users/items/accounts/transactions et recommande la récupération incrémentale des transactions via le paramètre `since` après réception des webhooks `item.account.updated`. Cette mécanique justifie un stockage en étages :

raw -> normalized -> classified -> ledgerized -> aggregated. ²³



Le schéma relationnel recommandé ci-dessous constitue le noyau de Ledger V2. Les noms sont proposés comme **cible de design** et peuvent être adaptés à la convention de nommage du dépôt une fois celui-ci accessible.

Table	Rôle	Clés / contraintes principales	Source / mode d'alimentation	Données dérivées
<code>workspaces</code>	Partition logique des données	<code>id</code> , <code>owner_user_id</code>	Création utilisateur	scope RLS
<code>bank_connections</code>	Connexions Bridge	<code>workspace_id</code> , <code>bridge_user_id</code> , <code>item_id</code> , <code>status</code> , <code>authentication_expires_at</code>	API Bridge / webhook	état connecteur
<code>accounts</code>	Comptes bancaires et internes	<code>provider_account_id</code> , <code>account_type</code> , <code>currency</code> , <code>is_internal</code>	Bridge / manuel	soldes
<code>import_batches</code>	Historisation des imports	<code>source_type</code> , <code>status</code> , <code>checksum</code> , <code>started_at</code> , <code>ended_at</code>	système	audit reprise
<code>raw_transactions</code>	Données bancaires brutes	<code>provider_tx_id</code> , <code>payload_json</code> , <code>fetches_at</code> , <code>updated_at</code>	Bridge / CSV	staging
<code>transactions</code>	Transactions normalisées	unicité (<code>workspace_id</code> , <code>dedupe_hash</code>) ; <code>direction</code> , <code>amount</code> , <code>booking_date</code> , <code>value_date</code>	pipeline normalization	base métier
<code>merchants</code>	Référentiel tiers	<code>normalized_name</code> , <code>fingerprint</code>	auto + manuel	enrichissement
<code>categories</code>	Nature de flux	hiérarchie, <code>code</code> , <code>parent_id</code>	référentiel	reporting
<code>function_buckets</code>	Rubriques de gestion	<code>code</code> , <code>report_axis</code>	référentiel	résultat/ trésorerie
<code>ledger_entries</code>	En-têtes d'écriture	<code>entry_type</code> , <code>source_object_type</code> , <code>source_object_id</code> , <code>balanced=true</code>	générateur ledger	journal
<code>ledger_lines</code>	Lignes débit/ crédit	contrainte <code>sum(debit)=sum(credit)</code> par entrée	générateur ledger	balance

Table	Rôle	Clés / contraintes principales	Source / mode d'alimentation	Données dérivées
subscriptions	Abonnements détectés ou manuels	merchant_id , expected_amount , periodicity , next_expected_date	IA + validation	prévision
subscription_occurrences	Occurrences attendues	subscription_id , due_date , expected_amount , status	scheduler	rapprochem
contracts	Contrats hors abonnement	kind , start_date , end_date , schedule_rule	manuel / import doc	échancier
forecast_scenarios	Scénarios	scenario_type , baseline_from_date , status	métier	simulations
forecast_entries	Écritures futures	scenario_id , entry_date , amount , confidence , source_rule	moteur forecast	cashflow
assets	Actifs patrimoniaux	asset_type , acquisition_date , gross_value	manuel	patrimoine
loans	Emprunts	principal , rate , term_months , amortization_type	manuel / import	échancier
patrimoine_entries	Historique patrimonial	origin_type , valuation_date , book_value , market_value	moteur patrimoine	net worth
tax_candidates	Candidats fiscaux	tax_code , confidence , review_status	moteur fiscal/ IA	dossier fiscal
documents	Pièces justificatives	storage_provider , bucket , object_key , checksum	upload	preuve
audit_logs	Traçabilité	actor , event_type , payload , correlation_id	système	conformité
asset_packs	Gouvernance modulaire	module_code , version , status , owner , compatibility	admin	marketplace interne

La gestion des données doit être gouvernée par une **matrice de vérité**.

Couche	Exemple	Mutable	Source de vérité	Recalculable
Source brute	<code>raw_transactions.payload_json</code>	Non, append-only	Oui	N/A
Métier normalisé	<code>transactions</code>	Oui, versionnée	Oui	Oui
Référentiel	<code>categories</code> , <code>function_buckets</code>	Oui	Oui	N/A
Dérivé comptable	<code>ledger_entries</code> , <code>ledger_lines</code>	Rebuild contrôlé	Oui	Oui
Dérivé analytique	vues KPI, monthly results	Non, recalcul	Non	Oui
Dérivé IA	suggestions, confidence	Oui, toujours révisable	Non	Oui
Audit	<code>audit_logs</code>	Non, append-only	Oui	Non

Les règles de calcul doivent être **formalisées**, versionnées et testées. Les algorithmes suivants sont recommandés comme baseline.

```

ALGORITHME NORMALISATION_TRANSACTION(raw_tx):
  input:
    raw_tx(provider, provider_tx_id, label, amount, currency, booking_date,
    value_date, account_id, payload_json)
  steps:
    1. canonical_label := trim(uppercase(remove_diacritics(label)))
    2. merchant_hint := merchant_normalizer(canonical_label)
    3. signed_amount := normalize_sign(provider, amount, payload_json)
    4. booking_date := coalesce(raw_tx.booking_date, raw_tx.date)
    5. value_date := coalesce(raw_tx.value_date, booking_date)
    6. direction := if signed_amount < 0 then "OUTFLOW" else "INFLOW"
    7. normalized_amount := abs(round_currency(signed_amount))
    8. dedupe_hash := sha256(workspace_id, account_id, provider_tx_id?,
    booking_date, normalized_amount, canonical_label)
    9. emit transaction_normalized

```

```

ALGORITHME DEDOUBLONNAGE(batch):
  for each tx in batch:
    if exists(transaction where provider_tx_id = tx.provider_tx_id and
    provider non null):
      mark tx as DUPLICATE_PROVIDER_ID
    else if exists(transaction where dedupe_hash = tx.dedupe_hash):
      mark tx as DUPLICATE_HASH
    else if fuzzy_match(tx, transactions within ±3 days and same abs(amount)
    and similar label > threshold):
      mark tx as POSSIBLE_DUPLICATE_REVIEW

```

```
else:
    accept tx
```

```
ALGORITHME CLASSIFICATION(transaction):
    features :=
        merchant_fingerprint,
        canonical_label_tokens,
        amount_band,
        iban_or_card_hint,
        account_type,
        periodicity_pattern,
        counterparty,
        historical_user_decisions

    rule_hit := evaluate_user_rules(features)
    if rule_hit.confidence >= 1.0:
        assign(rule_hit.category, rule_hit.function_bucket, source="RULE")
    else:
        ml_suggestion := ai_classifier(features)
        assign(ml_suggestion.category, ml_suggestion.function_bucket,
source="AI", confidence=ml_suggestion.score)

    if confidence < MIN_ACCEPT_THRESHOLD:
        status := TO_REVIEW
    else:
        status := CLASSIFIED
```

```
ALGORITHME GENERATION_LEDGER(transaction):
    assert transaction.status in ("CLASSIFIED","VALIDATED")
    lines := []

    if transaction.kind = INTERNAL_TRANSFER:
        lines += debit(destination_internal_account, amount)
        lines += credit(source_internal_account, amount)

    else if transaction.direction = OUTFLOW:
        lines += debit(expense_or_asset_account(transaction.mapping), amount)
        lines += credit(bank_account(transaction.account_id), amount)

    else if transaction.direction = INFLOW:
        lines += debit(bank_account(transaction.account_id), amount)
        lines += credit(income_or_liability_account(transaction.mapping),
amount)

    if transaction.fees > 0:
        split fee line according to fee policy
```

```

assert sum(debit(lines)) = sum(credit(lines))
persist ledger_entry + ledger_lines

```

```

ALGORITHME FORECAST(scenario, horizon):
    baseline := recurring_occurrences(subscriptions, contracts, loans, taxes,
goals, horizon)
    seasonality := derive_monthly_patterns(classified_transactions_history)
    manual_events := scenario.manual_events
    scenario_overrides := scenario.assumptions

    for each future_day in horizon:
        projected_cashflow[day] :=
            baseline[day]
            + seasonality_adjustments[day]
            + manual_events[day]
            + scenario_overrides[day]

    projected_balance[day] :=
        projected_balance[day-1] + projected_cashflow[day]

    emit forecast_entries and alert if projected_balance < threshold

```

```

ALGORITHME AMORTISSEMENT_LOAN(loan):
    if loan.amortization_type = "FRENCH":
        mensualite = pmt(loan.rate/12, loan.term_months, loan.principal)
        for month in 1..term_months:
            interest = outstanding * loan.rate/12
            principal = mensualite - interest
            outstanding = outstanding - principal
            emit schedule(month, principal, interest, outstanding)
    else if type = "LINEAR":
        principal = loan.principal / loan.term_months
        for month in 1..term_months:
            interest = outstanding * loan.rate/12
            outstanding = outstanding - principal
            emit schedule(month, principal, interest, outstanding)

```

```

ALGORITHME RAPPROCHEMENT_ABONNEMENT(subscription, transactions):
    expected := next_open_occurrences(subscription)
    candidates := transactions
        where merchant similarity >= threshold
        and amount within tolerance
        and booking_date within window(expected.due_date ± tolerance_days)

    score := weighted_similarity(merchant, label, amount, periodicity, account)
    best := argmax(score)

    if best.score >= AUTO_MATCH_THRESHOLD:

```

```

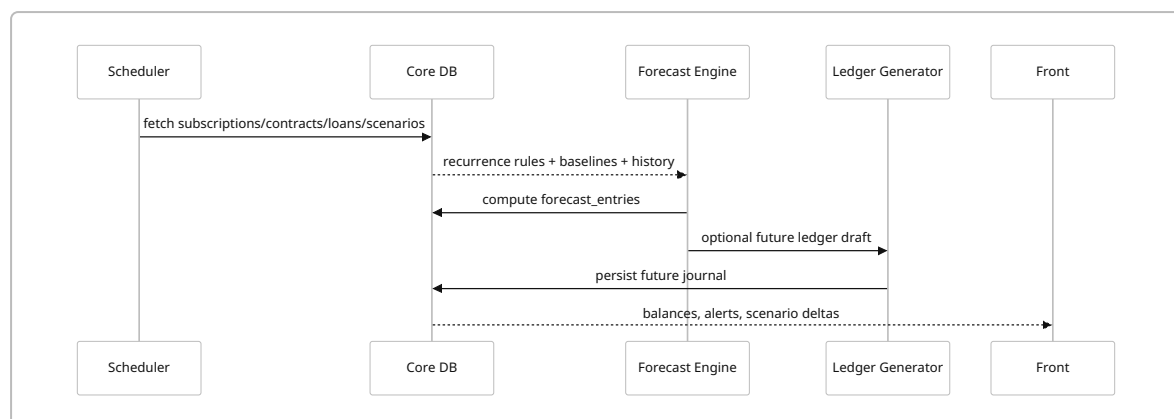
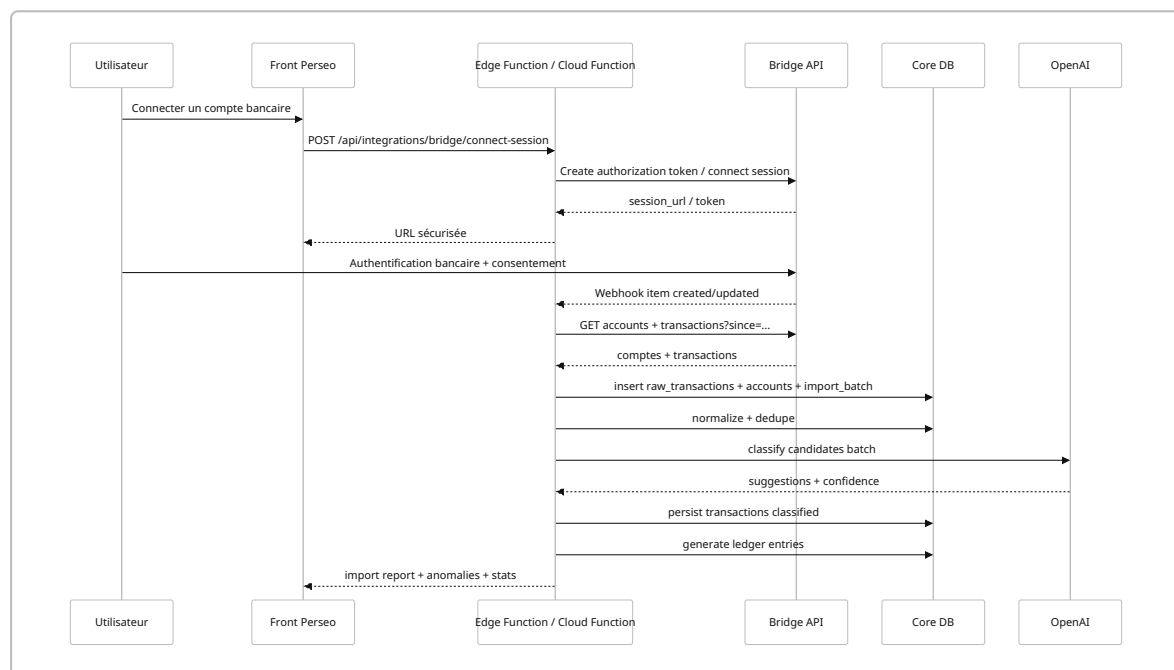
    reconcile(expected, best.transaction_id, status="AUTO_MATCHED")
else if best.score >= REVIEW_THRESHOLD:
    create_manual_review(expected, best)
else:
    keep_occurrence as MISSED_OR_PENDING

```

Le choix d'un **ledger équilibré** permet de garantir que les états analytiques peuvent être recalculés de manière déterministe. Ce choix est cohérent avec la logique de journal décrite par les systèmes de double écriture et avec le positionnement naturel de PostgreSQL pour les contraintes de cohérence et d'équilibrage. ²⁴

Flux applicatifs, API contractuelles et intégrations externes

Les flux à forte valeur doivent être implémentés comme des pipelines explicites et auditable. Bridge documente un modèle REST/JSON avec tokens d'autorisation utilisateur, comptes, transactions, webhooks, statuts d'items et gestion de la SCA ; Supabase et Firebase documentent les fonctions serveur HTTP/événementielles ; Stripe documente Checkout, Customer Portal, webhooks et test clocks ; OpenAI documente les Responses API, les bonnes pratiques de production et les limites de débit. ²⁵



Le flux d'import bancaire doit suivre ce contrat d'intégration : toute connexion bancaire est créée côté serveur, toute synchronisation est pilotée par webhook ou par job planifié, toute transaction est d'abord persistée en brut, puis normalisée, puis dédoublonnée, puis classifiée, puis ledgerisée. Bridge recommande explicitement l'usage des webhooks et de la récupération incrémentale via `since` après `item.account.updated`, et rappelle que les données reçues sont sensibles et doivent être protégées avec un haut niveau de sécurité. ²⁶

Les API internes de Perseo doivent être contractuelles, versionnées et documentées. Une proposition de surface API est donnée ci-dessous.

Endpoint	Méthode	Auth	Description	Réponse nominale	Erreurs attendues
<code>/api/integrations/bridge/connect-session</code>	POST	user JWT	Crée une session Bridge Connect	<code>{sessionId, expiresAt}</code>	400 , 401 , 409 , 502
<code>/api/integrations/bridge/webhook</code>	POST	signature secret	Reçoit événements Bridge	<code>{received:true}</code>	400 , 401 , 429 , 500
<code>/api/imports/csv</code>	POST multipart	user JWT	Upload CSV bancaire	<code>{batchId, stagedCount}</code>	400 , 413 , 422
<code>/api/transactions/classify</code>	POST	user JWT	Lance classification d'un lot	<code>{jobId, count}</code>	400 , 409 , 429
<code>/api/ledger/rebuild</code>	POST	admin/ workspace	Rebuild ledger sur périmètre	<code>{jobId, scope, estimatedImpact}</code>	400 , 409 , 423
<code>/api/forecast/run</code>	POST	user JWT	Recalcule un scénario	<code>{scenarioId, alerts, projectedBalances}</code>	400 , 422
<code>/api/subscriptions/reconcile</code>	POST	user JWT	Lance rapprochements	<code>{matched, review, missed}</code>	400 , 409
<code>/api/stripe/checkout</code>	POST	user JWT	Crée session Checkout	<code>{checkoutUrl}</code>	400 , 401 , 502
<code>/api/stripe/portal</code>	POST	user JWT	Ouvre Customer Portal	<code>{portalUrl}</code>	400 , 404 , 502

Endpoint	Méthode	Auth	Description	Réponse nominale	Erreurs attendues
<code>/api/ai/insights</code>	POST	user JWT	Génère narrative / insights	<code>{summary, model, usage}</code>	400, 429, 503

Des exemples de payloads recommandés :

```
{
  "workspaceId": "ws_123",
  "accountIds": ["acc_1", "acc_2"],
  "since": "2026-06-01T00:00:00Z",
  "mode": "incremental"
}
```

```
{
  "workspaceId": "ws_123",
  "scenarioId": "scn_base_2026_q3",
  "horizonDays": 180,
  "includeSubscriptions": true,
  "includeLoans": true,
  "includeManualEvents": true
}
```

```
{
  "workspaceId": "ws_123",
  "transactionIds": ["tx_1", "tx_2"],
  "provider": "openai",
  "purpose": "classification"
}
```

Le contrat d'erreur doit être uniforme, par exemple :

```
{
  "error": {
    "code": "BRIDGE_AUTH_EXPIRED",
    "message": "L'authentification bancaire nécessite une nouvelle SCA.",
    "retryable": true,
    "provider": "bridge",
    "correlationId": "corr_01J..."
  }
}
```

La sécurité d'API doit reposer sur quatre principes : **authentification utilisateur, autorisation workspace-scoped, signature webhook, idempotence**. Supabase indique que l'invocation d'Edge

Functions requiert un header d'autorisation ; Stripe demande la vérification des événements de webhook ; Bridge recommande l'usage du secret de webhook et l'IP allowlisting ; OpenAI recommande la protection des clés API et la gestion des rate limits. ²⁷

Concernant les intégrations externes, les règles de conception sont les suivantes. **Bridge** : serveur uniquement, consentement horodaté, suivi `authentication_expires_at`, resync incrémental, rejouabilité des webhooks. **Stripe** : Checkout/Portal/Webhooks, aucun stockage de PAN, tout flux carte délégué à Stripe pour limiter le périmètre PCI. **OpenAI** : appels uniquement depuis fonction serveur, prompts versionnés, redaction/pseudonymisation quand possible, budget d'usage et backoff sur 429. Ces choix sont alignés sur les recommandations officielles des fournisseurs. ²⁸

Sécurité, conformité, exploitation et gouvernance

Le cadre de sécurité doit être pensé dès le design. Le RGPD impose une base légale, la transparence, la minimisation, la protection des données by design/by default, la sécurité du traitement, la traçabilité des activités de traitement, le droit d'accès, d'effacement et la documentation des violations. Les articles 5, 6, 7, 13, 15, 17, 25, 30, 32 et 33 sont particulièrement structurants pour Perseo, qui traite des données bancaires et potentiellement des données fiscales ou documentaires sensibles au sens métier, même si elles ne relèvent pas nécessairement des catégories spéciales de l'article 9. ²⁹

La couche Open Banking doit être gouvernée en lien avec la **PSD2** et le règlement délégué sur la **SCA**. Bridge présente Bridge Connect comme une application conforme PSD2, documente la SCA, la gestion des items, l'expiration de l'authentification et la nécessité de sécuriser les flux webhook. Le système Perseo doit donc journaliser la date/heure de consentement, l'état de connexion, l'expiration de la SCA, les resynchronisations et la révocation. ³⁰

La facturation SaaS via Stripe engage aussi un cadre PCI. Le PCI SSC définit le PCI DSS comme le référentiel de base pour protéger les données de compte de paiement ; Stripe rappelle que la conformité PCI est une responsabilité partagée et que toute entité qui stocke, traite ou transmet des données de carte doit s'y conformer. La recommandation d'architecture est donc stricte : **Perseo ne doit jamais stocker ni faire transiter de PAN côté backend propre** ; tous les flux carte doivent passer par Checkout/Elements/Portal, avec stockage minimal des métadonnées strictement nécessaires au compte client. ³¹

Les exigences non fonctionnelles cibles, faute de SLA business spécifié, doivent être fixées comme **objectifs techniques recommandés** :

Domaine	Exigence recommandée	Justification
Disponibilité front	99,9 % cible	Cohérent avec site web de production ; non contractuel
Disponibilité flux bancaires	traitement différé accepté ; SLO 95 % < 15 min post-webhook	dépend des fournisseurs externes
Latence lecture dashboard	p95 < 400 ms côté API	UX analytique acceptable
Latence import/classification	asynchrone avec job status	traitement potentiellement lourd

Domaine	Exigence recommandée	Justification
RPO	≤ 15 min cible avec PITR si Supabase primaire ; sinon export planifié Firestore	compatible avec documents de backup et reprise ³²
RTO	≤ 4 h cible	recommandé, non spécifié
Chiffrement transport	TLS obligatoire partout	Vercel HTTPS par défaut, Firestore TLS in transit, SSL enforcement Supabase ³³
Chiffrement au repos	provider-managed keys minimum ; CMEK si besoin Firebase	Firestore supporte CMEK ; RGPD art. 32 évoque chiffrement ³⁴
Audit	append-only + correlation ID	conformité et forensic
Sauvegarde documents	backup séparé du storage	Supabase précise que les backups DB ne couvrent pas Storage ³⁵

La checklist sécurité/conformité minimale est la suivante.

Contrôle	Exigence
AuthN	MFA admin recommandée, rotation des sessions, séparation user/admin
AuthZ	scoping strict par workspace, RLS ou Security Rules
Secrets	jamais côté front ; uniquement env secrets Edge/Functions
Webhooks	signature vérifiée, idempotency key, replay protection
Open Banking	consentement horodaté, révocation, suivi SCA
RGPD	registre de traitement, politique de rétention, export/effacement, privacy notice
PCI	aucune donnée carte brute stockée hors Stripe
Journaux	audit métier + audit sécurité + journaux d'intégration
Données IA	pseudonymisation partielle, conservation limitée, opt-in explicite si nécessaire
Chiffrement	TLS en transit, chiffrement au repos provider, SSL enforcement DB
Sauvegarde	stratégie DB + stratégie storage + tests de restauration
Accès admin	bastion logique, rôles nominaux, revue périodique

La gouvernance technique doit aussi couvrir les **asset packs**. Chaque module réutilisable — “Transactions”, “Ledger Core”, “Forecast”, “Subscriptions”, “Tax Candidates”, “Bridge Connector”, “Stripe Billing”, “AI Classification” — doit être gouverné comme un artefact versionné avec owner, interfaces, dépendances, migration, tests, changelog et politiques de compatibilité.

Asset pack	Owner	Dépendances	Contrat de sortie	Versioning
transactions-core	Lead backend	accounts, imports, merchants	normalized transactions	semver
ledger-core-v2	Lead architecture/data	transactions-core, chart-of-accounts	balanced entries	semver strict
forecast-engine	Lead data/product	ledger-core-v2, subscriptions, loans	forecast entries + alerts	semver
bridge-adapter	Intégration	secrets, users, accounts	import batches	version API Bridge
stripe-billing	Platform	auth, plans, customers	subscription status	version API Stripe
ai-classification	AI/Backend	transactions-core, tax rules	suggestions scored	model + prompt version
tax-engine	Functional/data	ledger-core-v2, transaction tags	tax candidates/export	semver

Migration, stratégie de tests et roadmap technique

La migration vers **Ledger V2** doit être menée comme une migration de **modèle de vérité**, pas comme un simple refactoring de schéma. Supabase documente les migrations SQL et le CLI comme mécanisme standard de gestion de schéma ; il documente aussi les backups quotidiens, le PITR et la restauration vers un nouveau projet. Cela permet de formaliser une migration progressive, rejouable et réversible.

36

La stratégie de migration recommandée est la suivante :

Étape	Action	Résultat
Préparation	geler schéma existant, cataloguer tables et dépendances	cartographie “as-is”
Ajout parallèle	créer tables <code>ledger_entries_v2</code> , <code>ledger_lines_v2</code> , mapping accounts, vues compatibilité	coexistence V1/V2
Backfill	générer entries V2 depuis <code>transactions</code> historiques	journal parallèle
Validation	comparer agrégats V1/V2 par période, compte, catégorie	écarts identifiés
Double écriture	nouvelles transactions écrivent V1+V2 temporairement	stabilisation
Cutover lecture	tableaux de bord et prévisions lisent V2	bascule fonctionnelle
Décommission	gel V1 puis suppression différée	simplification

Exemple de plan de migration SQL cible :

```
-- 001_create_ledger_v2.sql
create table ledger_entries_v2 (
  id uuid primary key,
  workspace_id uuid not null,
  source_object_type text not null,
  source_object_id uuid not null,
  entry_date date not null,
  currency text not null,
  created_at timestamptz not null default now()
);

create table ledger_lines_v2 (
  id uuid primary key,
  ledger_entry_id uuid not null references ledger_entries_v2(id) on delete
cascade,
  account_code text not null,
  debit numeric(18,2) not null default 0,
  credit numeric(18,2) not null default 0,
  dimensions jsonb not null default '{}'
);

create index idx_ledger_entries_v2_workspace_date
  on ledger_entries_v2(workspace_id, entry_date);

create index idx_ledger_lines_v2_entry
  on ledger_lines_v2(ledger_entry_id);
```

```
-- 002_constraints_and_views.sql
create or replace view ledger_balance_check_v2 as
select
  e.id,
  round(sum(l.debit) - sum(l.credit), 2) as imbalance
from ledger_entries_v2 e
join ledger_lines_v2 l on l.ledger_entry_id = e.id
group by e.id;

-- invariant attendu: imbalance = 0
```

Le plan de rollback doit être documenté dès le départ : **pas de suppression immédiate de V1, double écriture transitoire, switch de lecture par feature flag, snapshot pré-migration, restauration PITR ou restauration vers projet de secours si échec majeur**. Supabase documente précisément la restauration à un instant et vers un nouveau projet ; Firestore documente l'export/import pour reprise ou migration croisée. ³⁷

La stratégie de tests doit couvrir six couches :

Couche	Outils / mode	Objectif
Tests de règles	unitaires purs	normalisation, dédoublonnage, mapping
Tests DB	SQL + jeux de données	contraintes, équilibrage, vues, RPC
Tests fonctions serveur	Deno test / Firebase emulator	webhooks, auth, secrets, erreurs
Tests d'intégration	sandbox Bridge, Stripe test mode, OpenAI mock	contrats fournisseurs
Tests E2E	navigateur + preview env	parcours utilisateur critiques
Tests de migration	replay historique + comparaison V1/V2	non-régression comptable

Supabase documente les tests d'Edge Functions et le développement local ; Firebase documente la Local Emulator Suite ; Stripe documente les test clocks pour simuler les cycles de souscription et les webhooks ; GitHub Actions documente les matrices pour multiplier les variantes de test. ³⁸

La roadmap technique priorisée peut être formulée ainsi.

Phase	Priorité	Livrables
Socle d'architecture	Critique	BackendManager, décision provider primaire, auth, RLS/Rules, secrets, audit, CI/CD, environnements
Ingestion & normalisation	Critique	Bridge adapter, imports CSV, staging brut, normalisation, dédoublonnage, référentiels marchands
Ledger V2	Critique	schéma V2, mapping comptes, générateur équilibré, vues de contrôle, rebuild job, backfill
Restitution analytique	Haute	dashboard, résultat, trésorerie, alertes, requêtes/vues optimisées
Engagements & prévisions	Haute	abonnements, contrats, occurrences, moteur forecast, scénarios, journal futur
Patrimoine & fiscalité	Haute	assets/loans, amortissements, tax candidates, exports
Industrialisation	Haute	observabilité, PRA, backup/restore drills, SAST/DAST, performance tests
Produit SaaS	Moyenne	Stripe checkout/portal, plans, contrôle d'usage, RGPD self-service, admin asset packs

En termes de décision finale, le cahier des charges technique peut être résumé ainsi : **Perseo doit être développé comme une plateforme web financière modulaire à noyau Ledger V2, avec Supabase/ PostgreSQL recommandé comme source de vérité primaire, Firebase conservé comme provider compatible si nécessaire, toutes les intégrations externes encapsulées en fonctions serveur, une séparation stricte entre données brutes, métiers et dérivées, et une gouvernance formelle des modules par asset packs.** Les briques techniques disponibles dans les documentations officielles

couvrent bien les besoins de sécurité, de déploiement, de migration, d'intégration et de reprise ; le seul point réellement indisponible à ce stade est la vérification directe du dépôt GitHub demandé, non accessible publiquement pendant la recherche. 39

- 1 2 10 39 **Row Level Security | Supabase Docs**
https://supabase.com/docs/guides/database/postgres/row-level-security?utm_source=chatgpt.com
- 3 **Internal Error**
- 4 22 36 **Database Migrations | Supabase Docs**
https://supabase.com/docs/guides/deployment/database-migrations?utm_source=chatgpt.com
- 5 **Art. 25 GDPR – Data protection by design and by default**
https://gdpr-info.eu/art-25-gdpr/?utm_source=chatgpt.com
- 6 **List items**
https://docs.bridgeapi.io/reference/getitems?utm_source=chatgpt.com
- 7 16 35 **Database | Supabase Docs**
https://supabase.com/docs/guides/database/overview?utm_source=chatgpt.com
- 8 **Integration security guide**
https://docs.stripe.com/security/guide?utm_source=chatgpt.com
- 9 24 **Database Functions | Supabase Docs**
https://supabase.com/docs/guides/database/functions?utm_source=chatgpt.com
- 11 **Firestore | Firebase - Google**
https://firebase.google.com/docs/firestore?utm_source=chatgpt.com
- 12 **Edge Functions | Supabase Docs**
https://supabase.com/docs/guides/functions?utm_source=chatgpt.com
- 13 **Cloud Functions for Firebase**
https://firebase.google.com/docs/functions?utm_source=chatgpt.com
- 14 **Storage | Supabase Docs**
https://supabase.com/docs/guides/storage?utm_source=chatgpt.com
- 15 **Cloud Storage for Firebase - Google**
https://firebase.google.com/docs/storage?utm_source=chatgpt.com
- 17 **Export and import data | Firestore - Firebase**
https://firebase.google.com/docs/firestore/manage-data/export-import?utm_source=chatgpt.com
- 18 **Firebase Hosting - Google**
https://firebase.google.com/docs/hosting?utm_source=chatgpt.com
- 19 **CLI Reference | Supabase Docs**
https://supabase.com/docs/reference/cli/introduction?utm_source=chatgpt.com
- 20 **Introduction to Firebase Local Emulator Suite**
https://firebase.google.com/docs/emulator-suite?utm_source=chatgpt.com
- 21 **Understanding GitHub Actions**
https://docs.github.com/articles/getting-started-with-github-actions?utm_source=chatgpt.com
- 23 **Authorization token**
https://docs.bridgeapi.io/reference/createuserauthtoken?utm_source=chatgpt.com

25 **Welcome**

https://docs.bridgeapi.io/docs/quickstart?utm_source=chatgpt.com

26 **Data fetching**

https://docs.bridgeapi.io/docs/data-fetching?utm_source=chatgpt.com

27 **JavaScript: Invokes a Supabase Edge Function.**

https://supabase.com/docs/reference/javascript/v1/functions-invoke?utm_source=chatgpt.com

28 **Manage your item lifecycle**

https://docs.bridgeapi.io/docs/manage-the-lifecycle?utm_source=chatgpt.com

29 **Art. 5 GDPR – Principles relating to processing of personal ...**

https://gdpr-info.eu/art-5-gdpr/?utm_source=chatgpt.com

30 **2015/2366 - EN - Payment Services Directive - EUR-Lex**

https://eur-lex.europa.eu/eli/dir/2015/2366/oj/eng?utm_source=chatgpt.com

31 **Data Security Standard (PCI DSS)**

https://www.pcisecuritystandards.org/standards/pci-dss/?utm_source=chatgpt.com

32 **Database Backups | Supabase Docs**

https://supabase.com/docs/guides/platform/backups?utm_source=chatgpt.com

33 **Encryption and TLS**

https://vercel.com/docs/cdn-security/encryption?utm_source=chatgpt.com

34 **Customer-managed encryption keys (CMEK) | Firestore**

https://firebase.google.com/docs/firestore/cmek?utm_source=chatgpt.com

37 **Restore to a new project | Supabase Docs**

https://supabase.com/docs/guides/platform/clone-project?utm_source=chatgpt.com

38 **Testing your Edge Functions | Supabase Docs**

https://supabase.com/docs/guides/functions/unit-test?utm_source=chatgpt.com